

Sesiunea 1: Fundamentele Calității și Mindset-ul de Tester

Capitolul 1: De ce este necesară testarea? (Misiunea și Obiectivele QA)	2
1. Nevoia de Testare: Complexitatea Umană și Software	2
2. Obiectivele Fundamentale ale Testării.....	3
3. Impactul Economic al Testării (Costul Eșecului)	3
💡 Exemple Practice pentru Discuție	4
Știați că...?	4
Poveste Aplicată: „Centura de Siguranță”	4
👤 Activitate de grup: „Ochiul de Tester”	5
Capitolul 2: Lanțul Calității (Eroare vs. Defect vs. Eșec).....	5
1. Eroarea (Error / Mistake)	5
2. Defectul (Defect / Bug / Fault)	5
3. Eșecul (Failure)	6
✍️ Lanțul de Cauzalitate: Cum se propagă greșeala?	6
💡 Analiza de profunzime: De ce contează distincția?	7
Știați că...?	7
Poveste Aplicată: „Rețeta de Prăjitură”	7
👤 Exercițiu de Gândire: „Identifică Lanțul”	8
Capitolul 3: Psihologia Testării și Nivelurile de Independență.....	9
1. Mindset-ul de Tester vs. Mindset-ul de Dezvoltator	9
2. Comunicarea și Diplomația în QA.....	10
3. Nivelurile de Independență ale Testării (ISTQB Standard)	10
4. Cele 7 Principii ale Testării (Sinteză din Studiu Individual)	11
💡 Exemple Practice pentru Discuție	11
Știați că...?	12
Poveste Aplicată: „Scriitorul și Corectorul”	12
👤 Exercițiu de Gândire: „Provocarea Curiozității”	13
Capitolul 4: Managementul Riscurilor în Testare (Produs vs. Proiect)	13
1. Ce este Riscul?	13

2. Riscul de Produs (Product Risk).....	13
3. Riscul de Proiect (Project Risk)	14
 Matricea de Priorizare a Testării	14
 Exemple Practice pentru Discuție	15
Știați că...?	15
Poveste Aplicată: „Podul peste Râu”	15
 Exercițiu de Gândire: „Sesiunea de Identificare”	16
Capitol Bonus: Testarea vs. Debugging și Rolul de Quality Advocate	17
1. Testarea vs. Debugging: Cine, Ce și Când?.....	17
A. Testarea (Activitatea QA-ului)	17
B. Debugging / Depanarea (Activitatea Dezvoltatorului)	17
 Fluxul de Colaborare: Cercul Calității	18
2. Testerul ca „Quality Advocate” (Avocatul Calității).....	18
 Exemple Practice pentru Discuție	18
Știați că...?	19
Poveste Aplicată: „Medicul vs. Chirurgul”	19
 Provocarea „Interviului”: Testare vs Debugging	20
Capitolul 5: Recapitulare, Concluzii și Misiunea Practică	21
1. Recapitulare Exhaustivă: Ce am pus în „trusa de scule”?.....	21
2. Concluziile Sesiunii: „Legile de Aur” QualiAdept.....	22
3. Tema de Casă: „Misiunea de Detectiv QA” 	22
Partea A: Analiza Lanțului de Eșec (Mapping).....	22
 Sfat de final de la echipa QualiAdept.....	23

Capitolul 1: De ce este necesară testarea? (Misiunea și Obiectivele QA)

 În acest capitol, nu ne limităm la definiții seci. Explorăm rațiunea de a exista a unui departament de QA și impactul pe care un tester îl are asupra succesului (sau eșecului) unei companii. Testarea nu este o activitate opțională la finalul proiectului; este „centura de siguranță” a întregului proces de dezvoltare.

1. Nevoia de Testare: Complexitatea Umană și Software

Software-ul este creat de oameni, pentru oameni. Din acest motiv, este inerent predispus la

imperfecțiune.

- **Limitele Umane:** Oricât de bun ar fi un programator, oboseala, presiunea timpului sau simpla neînțelegere a unei cerințe de business pot duce la greșeli.
- **Complexitatea Sistemelor:** Aplicațiile moderne sunt rețele interconectate. O modificare minoră într-un modul poate cauza un „efect de domino” și poate dărâma o funcționalitate complet diferită.
- **Mediul Variabil:** Software-ul rulează pe mii de tipuri de dispozitive, browsere și rețele. Ceea ce funcționează pe laptopul developerului poate eșua lamentabil pe telefonul unui client din altă parte a lumii.

2. Obiectivele Fundamentale ale Testării

Contrar mitului că testarea înseamnă „doar să găsești bug-uri”, obiectivele noastre sunt mult mai strategice:

- **Evaluarea produsului:** Verificăm dacă ceea ce s-a construit corespunde cu ceea ce s-a cerut (Cerințele de Business).
- **Prevenirea defectelor:** Prin implicarea QA-ului încă din faza de design, putem identifica ambiguități în cerințe înainte ca prima linie de cod să fie scrisă. (Testarea timpurie).
- **Reducerea riscului:** Nu putem garanta că software-ul are 0 bug-uri, dar putem reduce riscul ca un defect critic să ajungă la client.
- **Câștigarea încrederii:** Oferim stakeholderilor (manageri, clienți) o imagine clară și onestă asupra nivelului de calitate. Dacă testerul spune „e gata”, clientul poate dormi liniștit.
- **Respectarea conformității:** În domenii precum cel medical, bancar sau auto, testarea este obligatorie prin lege pentru a asigura siguranța utilizatorilor.

3. Impactul Economic al Testării (Costul Eșecului)

Există o corelație directă între momentul găsirii unui bug și costul reparării acestuia:

- **Găsit în faza de Analiză:** Costă 1€ (doar modificăm un text într-un document).
- **Găsit în faza de Programare:** Costă 10€ (programatorul își corectează codul pe loc).
- **Găsit în faza de QA:** Costă 100€ (necesită raportare, re-testare, timp de management).
- **Găsit de Client (Producție):** Poate costa milioane de euro (daune de imagine, procese legale, pierderea clienților).

Exemple Practice pentru Discuție

Scenariul A: Eroarea de 10 Centi

- **Context:** Un sistem de facturare rotunjește greșit cu 10 bani fiecare tranzacție.
- **Întrebare:** Pare o problemă minoră?
- **Analiză QA:** Dacă banca procesează 10 milioane de tranzacții pe zi, eroarea de „doar 10 bani” devine o pierdere de 1 milion de euro zilnic. Testarea nu verifică doar dacă butonul e albastru, ci și logica matematică din spate.

Scenariul B: Aplicația de Glovo/Uber la ore de vârf

- **Context:** Aplicația funcționează perfect la ora 10 dimineața. La ora 19:00, când mii de oameni comandă simultan, sistemul se blochează.
- **Analiză QA:** Aceasta este o problemă de **Performanță**. Testarea trebuie să prevadă „stresul” la care va fi supus produsul în viața reală.

Știați că...?

💡 În 1996, racheta Ariane 5 a explodat la 37 de secunde după lansare din cauza unei erori software de conversie a datelor (un număr pe 64 de biți a fost forțat într-un spațiu de 16 biți)? Pierderea a fost de **370 milioane de dolari**. Un simplu test de unitate ar fi putut preveni această catastrofă.

Poveste Aplicată: „Centura de Siguranță”

🔗 Imaginează-ți că ești inginerul de siguranță al unei fabrici de mașini.

- **Programatorii** sunt mecanicii care assemblează motorul și caroseria. Ei vor ca mașina să fie rapidă și frumoasă.
- **Testerul** este cel care pune mașina în „Crash Test”. Tu nu vrei să distrugi mașina pentru că ești rău, ci pentru că vrei să știi dacă airbag-ul se deschide **atunci când contează**.

Fără tine, mașina ar putea arăta impecabil în showroom, dar ar fi o capcană mortală pe autostradă. Misiunea ta nu este să „strici” mașina, ci să te asiguri că ea protejează viața utilizatorului în cele mai grele condiții.

Activitate de grup: „Ochiul de Tester”

Priviți obiectul de lângă voi (o cană, un pix, telefonul).

- Care este **funcția** lui principală? (**Happy Path**)
- Cum s-ar putea strica dacă este folosit greșit? (**Negative Testing**)
- Ce s-ar întâmpla dacă 100 de oameni ar încerca să îl folosească simultan? (**Load Testing**)

Capitolul 2: Lanțul Calității (Eroare vs. Defect vs. Eșec)

 În acest capitol, demontăm confuzia comună dintre termenii „bug”, „eroare” și „problemă”. Pentru un utilizator obișnuit, aplicația „nu merge”. Pentru un tester QualiAdept, există o cauză, o manifestare și un rezultat. Înțelegerea acestui lanț este esențială pentru a scrie rapoarte de bug profesionale și pentru a ajuta programatorii să găsească sursa problemei.

1. Eroarea (Error / Mistake)

 **Definiție:** O acțiune umană care produce un rezultat incorect.

- **Sursa:** Creierul uman. Este o greșeală de judecată, de tastare sau de interpretare.
- **Cauze frecvente:**
 - **Presiunea timpului:** „Trebuie să terminăm până vineri!” (duce la neatenție).
 - **Cod complex:** Tehnologii noi sau logici matematice greu de urmărit.
 - **Comunicare deficitară:** Programatorul a înțeles că butonul trebuie să fie „Verde”, dar clientul voia „Albastru”.
 - **Lipsa de experiență:** Necunoașterea limbajului de programare sau a domeniului (ex: un programator care scrie software medical fără să înțeleagă termenii clinici).

2. Defectul (Defect / Bug / Fault)

 **Definiție:** Manifestarea statică a unei erori în software sau în documentație.

- **Sursa:** Codul sursă, specificațiile sau designul. Este „greșeala” care stă scrisă acolo,

așteptând să fie executată.

- **Unde se găsește:**
 - În liniile de cod (ex: o formulă de calcul greșită).
 - În cerințele de business (ex: un document care spune că utilizatorul poate avea vârsta de -5 ani).
 - În baza de date (ex: o coloană care lipsește).
- **Notă importantă:** Un defect poate exista în sistem zeci de ani fără să fie descoperit niciodată, dacă acea porțiune de cod nu este niciodată rulată de un utilizator.

3. Eșecul (Failure)

 **Definiție:** Deviația componentului sau a sistemului de la livrarea serviciului așteptat.

- **Sursa:** Interacțiunea utilizatorului cu sistemul în timpul rulării.
- **Manifestare:** Ceea ce vedem noi pe ecran: un mesaj de eroare, un ecran albastru, o sumă calculată greșit sau aplicația care se închide brusc.
- **Relația cu Defectul:** Un eșec este cauzat întotdeauna de un defect. Totuși, un eșec poate fi cauzat și de factori externi (ex: cade curentul, se întrerupe internetul, radiații electromagnetice care corup datele), fără să existe neapărat un bug în cod.

Lanțul de Cauzalitate: Cum se propagă greșeala?

Procesul urmează o linie logică strictă:

Omul (face o Eroare) → Introduce un Defect (în cod) → Utilizatorul rulează codul și vede un Eșec.

Exemplu complet:

- **Eroare:** Un programator obosit confundă operatorul > (mai mare) cu >= (mai mare sau egal) când scrie codul pentru verificarea vârstei legale de vot.
- **Defect:** În codul sursă rămâne scris: `if (age > 18) { canVote = true; }.`
- **Eșec:** Un tânăr de fix 18 ani intră pe site, își introduce vârsta, dar sistemul îi spune că nu are drept de vot (deși legal ar fi trebuit să aibă).

Analiza de profunzime: De ce contează distincția?

Ca tester, nu te oprești la „nu merge”. Tu faci **RCA (Root Cause Analysis - Analiza Cauzei Rădăcină)**.

- Dacă raportezi doar **Eșecul** („Nu pot vota la 18 ani”), programatorul trebuie să caute singur acul în carul cu fân.
- Dacă raportezi **Defectul** („Condiția de vârstă nu include valoarea 18”), programatorul repară bug-ul în 10 secunde.

Știați că...?

💡 Nu toate defectele produc eșecuri?

Gândește-te la o roată de rezervă a unei mașini care are o gaură (Defect). Atâta timp cât nu faci pană și nu folosești roata, mașina merge perfect (nu ai un Eșec). Dar în secunda în care încerci să o folosești, sistemul tău „eșuează”. În software, acesta se numește **Dead Code** sau porțiuni de cod care se activează doar în condiții rare (ex: doar în ziua de 29 februarie).

Poveste Aplicată: „Rețeta de Prăjitură”

🧠 Imaginează-ți că vrei să faci o prăjitură după o rețetă dintr-o carte.

1. **Eroarea:** Autorul cărții a fost distras când a scris rețeta și a scris „1 kg de sare” în loc de „1 linguriță de sare”.
2. **Defectul:** În cartea ta de bucate (care este „Codul”), stă scris acum un ingredient greșit. Cartea stă în bibliotecă, nu deranjează pe nimeni.
3. **Eșecul:** Tu începi să gătești. Pui kilogramul de sare, bagi la cuptor și la final obții o prăjitură necomestibilă.

Întrebare pentru cursant: Unde ar fi trebuit să intervină testerul pentru a economisi banii pe ingrediente?

- **Răspuns:** La faza de **Testare Statică**, citind rețeta înainte de a începe să gătească și observând că 1kg de sare este un **Defect** logic.

Exercițiu de Gândire: „Identifică Lanțul”

Analizează următoarea situație și identifică Eroarea, Defectul și Eșecul:

„Un inginer a configurat greșit serverul de mail, punând o adresă de destinatar greșită în fișierul de setări. În fiecare lună, facturile clienților ajung la o persoană necunoscută, iar compania nu își primește banii.”

1. **Eroarea este:** _____
2. **Defectul este:** _____
3. **Eșecul este:** _____

Capitolul 3: Psihologia Testării și Nivelurile de Independență

🔗 Succesul unui proces de testare depinde în mare măsură de factorul uman. Testarea poate fi percepută ca o activitate „negativă” (căutăm defecte, criticăm munca altora), dar în realitate este o activitate constructivă, de protejare a produsului. În acest capitol, explorăm cum trebuie să gândească un QA, cum trebuie să comunice și de ce este vital ca persoana care testează să fie diferită de cea care a scris codul.

1. Mindset-ul de Tester vs. Mindset-ul de Dezvoltator

Deși ambele roluri colaborează pentru același obiectiv (livrarea unui produs bun), ele necesită mentalități distincte și, uneori, opuse.

- **Mindset-ul de Dezvoltator (Creatorul):**

- **Scop:** Să construiască funcționalitatea.
- **Viziune:** „Cum pot face acest buton să salveze datele în baza de date?”
- **Bariera Psihologică:** Este dificil să îți găsești propriile greșeli. Creierul nostru are tendința de a vedea ceea ce „crede” că a scris, nu ceea ce a scris în realitate (**Confirmation Bias**).

- **Mindset-ul de Tester (Detectivul):**

- **Scop:** Să verifice reziliența și conformitatea.
- **Viziune:** „Ce se întâmplă dacă apăs butonul de 100 de ori?”, „Ce se întâmplă dacă deconectez internetul în timp ce se salvează datele?”.
- **Trăsături Cheie:**
 - **Curiozitate:** Dorința de a explora colțurile ascunse ale aplicației.
 - **Pesimism Profesional:** Presupunerea că „există un bug undeva, trebuie doar să-l găsesc”.
 - **Ochi Critic:** Atenția la detalii care par nesemnificative (un pixel deplasat, o secundă

de întârziere).

- **Atenție la detalii:** Abilitatea de a observa discrepanțe între cerințe și realitate.

2. Comunicarea și Diplomația în QA

Un tester care găsește 100 de bug-uri, dar nu știe să le comunice, va crea tensiuni în echipă. Raportarea unui defect este, în esență, transmiterea unei vești proaste. De aceea, comunicarea trebuie să fie:

- **Obiectivă, nu personală:** Nu spunem „Ai greșit aici”, ci „Sistemul se comportă diferit față de specificații în acest scenariu”.
- **Bazată pe fapte:** Raportul de bug trebuie să conțină pași clari, dovezi (screenshot-uri/log-uri) și rezultatul așteptat.
- **Diplomată:** Testerul și Dezvoltatorul nu sunt inamici. Ei sunt ca un pilot și un copilot. Dacă unul greșește, amândoi se prăbușesc.
- **Orientată pe soluție, nu pe vină:** Scopul este repararea defectului, nu găsirea vinovatului.

3. Nivelurile de Independență ale Testării (ISTQB Standard)

Independența crește eficacitatea testării. Cu cât persoana care testează este mai „departe” de cel care a creat codul, cu atât va găsi mai multe defecte.

- **Nivelul 0 (Fără independență):** Autorul își testează singur codul. (Cel mai rapid, dar cel mai puțin eficient – Confirmarea erorilor este aproape imposibilă).
- **Nivelul 1:** Testarea este făcută de o altă persoană din aceeași echipă (ex: un alt dezvoltator – **Peer Review**).
- **Nivelul 2:** O echipă de testare independentă în cadrul aceleiași organizații (QA specializați care cunosc produsul, dar nu au scris codul).
- **Nivelul 3 (Maximă independență):** Tester externi (companii de consultanță sau audit). Aceștia vin cu o perspectivă complet nouă, fără prejudecăți despre „cum ar trebui să meargă”.

Avantajele independenței: Testerii independenți văd defectele pe care creatorii le ignoră din obișnuință.

Dezavantajele independenței: Poate apărea o barieră de comunicare sau o întârziere în fluxul de lucru (QA-ul nu știe contextul tehnic la fel de bine ca dev-ul).

4. Cele 7 Principii ale Testării (Sinteză din Studiu Individual)

Psihologia testării este ghidată de aceste reguli de aur care ne temperează așteptările:

- **Testarea arată prezența defectelor, nu absența lor:** Poți dovedi că există bug-uri, dar nu poți dovedi niciodată că „nu mai există niciunul”.
- **Testarea exhaustivă este imposibilă:** Nu poți testa toate combinațiile de date. Trebuie să fii strategic.
- **Testarea timpurie economisește timp și bani:** QA-ul trebuie să înceapă odată cu cerințele.
- **Gruparea defectelor:** De obicei, 80% din probleme se află în 20% din module (**Principiul lui Pareto**).
- **Paradoxul Pesticidelor:** Dacă rulezi aceleași teste mereu, ele nu vor mai găsi bug-uri noi. Testele trebuie actualizate.
- **Testarea depinde de context:** O aplicație bancară se testează diferit de un joc pe telefon.
- **Eroarea „Absenței erorilor”:** Degeaba ai un soft fără bug-uri dacă nu este util utilizatorului sau nu respectă cerințele.

💡 Exemple Practice pentru Discuție

Scenariul A: „Ochiul Format”

- **Context:** Un programator lucrează 14 ore la un modul de plată. Este mândru că totul compilează.
- **Intervenția QA:** Testerul intră și, în loc să introducă suma „100”, introduce „ABC” sau „-50”. Sistemul crapă.
- **Analiză:** Programatorul s-a concentrat pe **cum ar trebui** să meargă. Testerul s-a concentrat pe **cum poate fi păcălit** sistemul.

Scenariul B: Tensiunea în echipă

- **Context:** Un tester trimite un mail tuturor (inclusiv managerilor): „Codul lui Ion este plin de erori groaznice, aplicația e un dezastru”.
- **Întrebare:** Ce principiu de comunicare a fost încălcat?
- **Analiză:** A personalizat eroarea și a creat un mediu ostil. Un tester QualiAdept ar fi raportat bug-urile individual în Jira, descriind impactul tehnic, nu calitatea colegului.

Știați că...?

💡 Există un concept numit „**Blindness to Change**” (Orbirea la schimbare)? Dacă te uiți la un ecran mult timp, creierul tău începe să ignore micile diferențe. De aceea, testerii fac pauze regulate sau schimbă modulele între ei. O pereche nouă de ochi va vedea mereu bug-ul pe care tu l-ai „ignorat” în ultimele 2 ore.

Poveste Aplicată: „Scriitorul și Corectorul”

💰 Imaginează-ți un scriitor care tocmai a terminat un roman de 500 de pagini.

Scriitorul a citit textul de 10 ori. El „știe” ce scrie în fiecare frază. Când ajunge la o pagină unde a scris din greșeală „mere” în loc de „mure”, creierul lui va citi automat „mure”, pentru că asta a vrut să scrie.

Corectorul (Testerul) primește cartea pentru prima dată. El nu are nicio conexiune emoțională cu textul. El citește literă cu literă. El va vedea imediat că „mere” nu are sens în contextul unei păduri de munte.

Morala: Indiferent cât de inteligent este scriitorul (Dezvoltatorul), el are nevoie de cineva cu un mindset diferit (Testerul) pentru a se asigura că mesajul (Aplicația) ajunge la cititor (Client) fără greșeli care să-i strice experiența.

🧠 Exercițiu de Gândire: „Provocarea Curiozității”

Avem un câmp de introducere a vârstei pe un site. Majoritatea oamenilor scriu „25”, „30”, „45”.

Scrie o listă de 5 „valori ciudate” pe care un tester curios le-ar introduce pentru a vedea dacă sistemul rezistă. (Ex: 0, 150, „douăzeci”, spațiu gol, -1).

Explicați la curs: De ce am ales aceste valori? (**Răspuns:** Testarea limitelor și a tipurilor de date).

Capitolul 4: Managementul Riscurilor în Testare (Produs vs. Proiect)

🎯 În lumea ideală, am avea timp infinit să testăm totul. În lumea reală, avem termene limită (deadlines), bugete fixe și resurse limitate. De aceea, testarea se bazează pe **Risc**. Un tester QualiAdept nu testează la întâmplare; el testează zonele unde probabilitatea unui eșec este cea mai mare și unde impactul aceluși eșec ar fi cel mai dureros pentru business.

1. Ce este Riscul?

Definiție: Un eveniment viitor, incert, care are un impact negativ asupra obiectivelor.

Riscul este definit de doi parametri esențiali:

- **Probabilitatea (Likelihood):** Cât de șanse sunt ca problema să apară?
- **Impactul (Impact):** Cât de tare ne va dura dacă problema apare?

Formula Riscului: $Risc = Probabilitate \times Impact$

Ca QA, misiunea noastră este să identificăm aceste riscuri din timp și să folosim testarea ca pe o metodă de **mitigare** (reducere) a lor.

2. Riscul de Produs (Product Risk)

Acesta se referă la posibilitatea ca software-ul să nu îndeplinească așteptările utilizatorului sau cerințele de business. Este riscul legat de „ceea ce livrăm”.

Exemple de Riscuri de Produs:

- **Defecțiuni funcționale:** Software-ul nu face ce ar trebui (ex: butonul de „Finalizează Comanda” nu reacționează).
- **Calitate non-funcțională scăzută:** Aplicația este prea lentă (Performanță), este greu de înțeles (Uzabilitate) sau este vulnerabilă la hackeri (Securitate).
- **Incapacitatea de a îndeplini cerințele:** Produsul funcționează tehnic, dar nu rezolvă nevoia clientului.
- **Defecte care duc la failure:** Prezența unor bug-uri critice care blochează utilizarea sistemului.

Rolul QA: Identificăm riscurile de produs prin analize de cerințe și testare intensivă pe zonele critice (ex: procesarea plăților într-un magazin online).

3. Riscul de Proiect (Project Risk)

Acesta se referă la factorii externi sau procesele interne care pot împiedica echipa să livreze produsul la timp sau în buget. Este riscul legat de „cum lucrăm”.

Exemple de Riscuri de Proiect:

- **Factori organizaționali:** Lipsa de personal (QA-ul pleacă în concediu sau demisionează), lipsa de training (echipa nu știe să folosească Jira).
- **Probleme tehnice de infrastructură:** Mediul de testare nu este gata, serverele cad mereu, nu avem acces la baza de date.
- **Probleme de management/furnizori:** Un partener extern nu livrează API-ul la timp, cerințele se schimbă în fiecare zi (Scope Creep).

Rolul QA: Deși nu controlăm direct riscurile de proiect, trebuie să le raportăm. Dacă mediul de testare cade zilnic, acesta este un risc de proiect care va întârzia testarea produsului.

Matricea de Prioritizare a Testării

Un tester inteligent folosește o matrice pentru a decide ce testează prima dată:

Probabilitate / Impact	Impact Scăzut	Impact Ridicat
Probabilitate Ridicată	Testăm dacă avem timp	PRIORITATE MAXIMĂ (Testăm imediat)
Probabilitate Scăzută	Ignorăm / Testăm la final	Testăm cu atenție

Exemple Practice pentru Discuție

Scenariul A: Aplicația Bancară (Risc de Produs)

- **Risc:** Există o șansă ca transferul de bani să fie procesat de două ori.
- **Analiză:** Probabilitatea este medie (ține de cod), dar **Impactul** este uriaș (pierderi financiare, procese).
- **Decizie QA:** Acest scenariu este prioritatea 1. Nu lansăm aplicația până nu suntem 200% siguri că tranzacția este unică.

Scenariul B: Lansarea de Crăciun (Risc de Proiect)

- **Risc:** Magazinul online trebuie lansat pe 1 Decembrie, dar echipa de design întârzie cu 2 săptămâni.
- **Analiză:** Acesta este un risc de proiect. Timpul de testare se comprimă.
- **Decizie QA:** QA-ul trebuie să avertizeze managementul că reducerea timpului de testare crește riscul ca bug-uri critice să ajungă la client.

Știați că...?

💡 În faza de planificare, testerii participă la sesiuni de **Risk-Based Testing (RBT)**? În aceste ședințe, toată echipa pune pe masă „ce ne sperie cel mai tare”. Zonele care îi sperie cel mai tare pe stakeholderi sunt cele care vor primi cele mai multe scenarii de testare.

Poveste Aplicată: „Podul peste Râu”

💭 Imaginează-ți că ești inspectorul de siguranță al unui pod nou construit.

- **Riscul de Produs:** Te întrebi: „Sunt cablurile destul de groase?”, „Rezistă pilonii la cutremur?”, „Asfaltul se crapă la îngheț?”. Acestea sunt legate de construcția în sine. Dacă pilonul cedează, podul cade (Eșec critic).
- **Riscul de Proiect:** Te întrebi: „Avem destui muncitori să terminăm înainte de inundațiile de toamnă?”, „Ne livrează fabrica de oțel materialul la timp?”, „Avem buget să plătim macaraua?”. Dacă bugetul se termină, podul nu va fi gata niciodată.

Concluzia QA: Pentru utilizatorul care vrea să traverseze (Clientul), nu contează de ce podul nu este sigur sau gata. El vrea un pod funcțional. Misiunea ta este să monitorizezi ambele tipuri de riscuri pentru a te asigura că, la final, trecerea peste râu se face în siguranță.

🧐 Exercițiu de Gândire: „Sesiunea de Identificare”

Avem un proiect de lansare a unei aplicații de livrare de mâncare (similar Glovo). Identifică și clasifică următoarele situații:

- Serverul pe care testăm este foarte vechi și se restartează singur. (**Risc de:** _____)

- Dacă un client comandă mâncare de 500 RON, aplicația aplică greșit reducerea de 10%.
(Risc de: _____)
- Singurul tester din echipă nu știe cum să folosească aplicația de mobil, doar cea de web.
(Risc de: _____)
- Când telefonul trece din 4G în Wi-Fi, comanda se pierde. (Risc de: _____)

Capitol Bonus: Testarea vs. Debugging și Rolul de Quality Advocate

 De multe ori, persoanele din afara industriei (și chiar unii începători) confundă testarea cu repararea codului. „Dacă ești tester, înseamnă că reperi bug-urile, nu?” – este întrebarea

clasică. În acest capitol bonus, vom delimita clar responsabilitățile și vom explora dimensiunea etică a profesiei de QA.

1. Testarea vs. Debugging: Cine, Ce și Când?

Deși sunt activități strâns legate, ele au scopuri și executanți diferiți. Este crucial să înțelegem unde se termină munca testerului și unde începe cea a dezvoltatorului.

A. Testarea (Activitatea QA-ului)

- **Scop:** Identificarea eșecurilor (failures) și verificarea conformității cu cerințele.
- **Proces:** Testerul rulează software-ul, observă comportamentul acestuia și raportează discrepanțele.
- **Rezultat:** Un raport de bug care descrie **ce** nu funcționează și în ce condiții.
- **Mindset:** „Demonstrez că sistemul nu se comportă cum ar trebui.”

B. Debugging / Depanarea (Activitatea Dezvoltatorului)

- **Scop:** Identificarea cauzei rădăcină (root cause) a unui defect, izolarea acestuia și eliminarea lui (repararea).
- **Proces:** Programatorul analizează codul, folosește unelte speciale (debuggers), urmărește log-urile de sistem și face modificări în fișierele sursă pentru a corecta eroarea.
- **Rezultat:** Cod corectat și o nouă versiune a aplicației gata pentru re-testare.
- **Mindset:** „Găsesc motivul tehnic pentru care a apărut eroarea și îl repar.”



Fluxul de Colaborare: Cercul Calității

- **Testerul** găsește un eșec (Failure).
- **Testerul** scrie un Bug Report (identifică Defectul).
- **Dezvoltatorul** preia raportul și începe procesul de **Debugging** pentru a găsi sursa tehnică.
- **Dezvoltatorul** repară codul.
- **Testerul** re-testează (Confirmation Testing) pentru a se asigura că defectul a dispărut și nu au apărut altele noi.

2. Testerul ca „Quality Advocate” (Avocatul Calității)

Un concept avansat în modelul QualiAdept este acela că testerul nu este doar un „găsitor de bug-

uri”, ci vocea clientului în interiorul echipei.

- **Etica Profesională:** Testerul are responsabilitatea morală de a raporta orice risc, chiar și atunci când există presiune din partea managementului pentru a lansa produsul mai repede.
- **Curajul de a spune „Nu”:** Uneori, QA-ul este cel care trebuie să spună: „Acest produs nu este sigur pentru utilizatori”, protejând astfel reputația companiei și siguranța clienților.
- **Empatia față de utilizator:** Testerul trebuie să se întrebe mereu: „Dacă mama mea sau copilul meu ar folosi această aplicație, ar fi frustrați? Ar fi în pericol?”.

Exemple Practice pentru Discuție

Scenariul A: „Pasează responsabilitatea”

- **Context:** Un dezvoltator îi spune testerului: „Am reparat bug-ul, poți să intri tu în cod să verifici dacă e bine scris?”.
- **Analiză QA:** Acesta este un moment critic. Testerul trebuie să refuze politicos verificarea codului sursă (dacă facem Blackbox testing) și să insiste pe verificarea comportamentului funcțional. Debugging-ul și verificarea calității codului la nivel de sintaxă este treaba programatorului.

Scenariul B: Bug-ul „Minor” cu impact Major

- **Context:** Există un bug care face ca textul unei pagini să se suprapună pe ecranele mici. Managerul spune: „E doar vizual, lasă-l așa, avem nevoie de lansare!”.
- **Perspectiva Avocatului Calității:** Testerul trebuie să explice impactul: „Dacă textul se suprapune, butonul de 'Confirmare Plată' devine inaccesibil pe 40% din telefoanele utilizatorilor noștri. Nu este doar un bug vizual, este un blocaj de business”.

Știați că...?

 Există o etapă numită **Unit Testing** unde dezvoltatorii fac „testare” pe bucăți mici de cod înainte să ajungă la QA? Deși se numește testare, ea este mai aproape de debugging, deoarece este făcută de autorul codului pentru a-și valida propria logică. Totuși, testarea de sistem (făcută de voi, QA-ul) rămâne singura care garantează că toate piesele puzzle-ului se potrivesc în lumea reală.

Poveste Aplicată: „Medicul vs. Chirurgul”

● Imaginează-ți un spital modern.

- **Testerul este Medicul Diagnostician.** Tu ești cel care îl consultă pe pacient (Software-ul). Faci analize, observi simptomele (Eșecurile) și pui un diagnostic: „Pacientul are o problemă la inimă” (Bug Report). Tu nu intri în operație, dar ești cel care a salvat viața pacientului pentru că ai observat problema la timp.
- **Dezvoltatorul este Chirurgul.** El preia diagnosticul tău, deschide pacientul (Codul), caută exact unde este problema (Debugging) și face intervenția necesară pentru a repara organul bolnav.

Morala: Chirurgul (Dev) nu poate opera fără un diagnostic corect (QA), iar diagnosticul Medicului (QA) nu are valoare dacă pacientul nu este operat (Dev). Ambele roluri sunt vitale, dar sunt profesii diferite. Medicul (Testerul) veghează asupra sănătății generale a pacientului pe termen lung, fiind „Avocatul Sănătății” acestuia.

Provocarea „Interviului”: Testare vs Debugging

Dacă la un interviu ești întrebat: „De ce nu repară testerii bug-urile, dacă tot le găsesc?”, care ar fi răspunsul tău acum?

- **Răspuns ideal:** „Din motive de independență și eficiență. Un tester are un mindset critic, orientat pe ansamblu și pe experiența utilizatorului, în timp ce repararea necesită un mindset constructiv și o cunoaștere intimă a arhitecturii codului. În plus, dacă aș repara eu bug-ul, aș cădea în capcana 'Confirmation Bias', pierzându-mi obiectivitatea necesară pentru a valida soluția ulterior.”

Capitolul 5: Recapitulare, Concluzii și Misiunea Practică

💰 O sesiune QualiAdept nu este completă până când nu ne asigurăm că informația a trecut de la „am auzit” la „am înțeles cum să aplic”. Acest capitol final servește drept busolă pentru tot ce am construit astăzi: de la necesitatea economică a testării până la psihologia subtilă din spatele fiecărui defect raportat.

1. Recapitulare Exhaustivă: Ce am pus în „trusa de scule”?

Astăzi am explorat cinci piloni fundamentali care definesc profesia de QA Manual:

- **Pilonul 1: Necesitatea Testării.** Am înțeles că software-ul este imperfect deoarece este o creație umană. Testarea nu este un lux, ci o metodă de supraviețuire a business-ului, reducând costurile enorme ale defectelor găsite prea târziu.
- **Pilonul 2: Lanțul Calității (Terminologie).** Am definit clar triada:
 - **Eroarea:** Greșeala din mintea sau mâna omului.
 - **Defectul (Bug-ul):** Greșeala „înghețată” în cod sau documentație.
 - **Eșecul (Failure):** Momentul în care utilizatorul observă că sistemul nu funcționează.
- **Pilonul 3: Psihologia și Independența.** Am învățat că un tester trebuie să fie un „pesimist profesionist” și un „detectiv curios”. Am văzut de ce independența (persoana care testează să fie diferită de cea care scrie) crește exponențial șansele de a găsi erori critice.
- **Pilonul 4: Managementul Riscurilor.** Am învățat să prioritizăm. Nu avem timp să testăm tot, așa că testăm unde probabilitatea este mare și impactul este devastator (Risc de Produs) și monitorizăm factorii care pot întârzia livrarea (Risc de Proiect).
- **Pilonul Bonus: Testare vs. Debugging.** Am delimitat responsabilitățile. QA-ul diagnostichează (identifică failure-ul), iar Dezvoltatorul operează (repară defectul în cod).

2. Concluziile Sesiunii: „Legile de Aur” QualiAdept

Dacă ar fi să pleci cu doar câteva idei după această sesiune, acestea ar trebui să fie:

- **Testarea nu garantează absența bug-urilor.** Ea demonstrează doar prezența celor pe care am reușit să le găsim. Un soft „fără bug-uri” este un soft care nu a fost testat destul.
- **Testarea timpurie este cea mai ieftină.** Cu cât găsim problema mai aproape de faza de idee (cerințe), cu atât salvăm mai mulți bani și timp.
- **Comunicarea este la fel de importantă ca tehnica.** Un bug raportat agresiv va fi respins; un bug raportat diplomatic și argumentat va fi reparat cu prioritate.
- **Ești avocatul utilizatorului.** Misiunea ta este să te asiguri că experiența clientului final este fluidă și sigură.

3. Tema de Casă: „Misiunea de Detectiv QA” 🚀

Această temă este concepută pentru a-ți antrena „ochiul de tester” pe scenarii reale. Te rugăm să redactezi răspunsurile într-un document (Word/PDF) și să le încarci în folderul tău personal până la sesiunea următoare.

Partea A: Analiza Lanțului de Eșec (Mapping)

 Analizează următorul scenariu:

„O companie aeriană lansează o actualizare pentru sistemul de check-in. Din cauza unei neînțelegeri între analist și programator, sistemul permite pasagerilor să rezerve locuri cu numărul 0, deși avionul începe de la rândul 1. La poarta de îmbarcare, 20 de pasageri nu pot urca în avion deoarece locurile lor nu există fizic.”

- Identifică **Eroarea umană**.
- Identifică **Defectul (Bug-ul)** din sistem.
- Identifică **Eșecul (Failure-ul)** observat de pasageri.
- Explică: Cum ar fi putut un tester să prevină acest lucru în faza de **Testare Statică** (analiza cerințelor)?

Partea B: Evaluarea Riscurilor (Prioritizare)

Imaginează-ți că ești QA pentru o aplicație de tip **Mobile Banking** (ex: Revolut/BT Pay). Trebuie să testezi două funcționalități noi, dar ai timp doar pentru una singură înainte de lansare:

- **Funcționalitatea 1:** Schimbarea culorii de fundal a profilului utilizatorului (Personalizare).
- **Funcționalitatea 2:** Transferul de bani prin număr de telefon (Plăți).
- Clasifică cele două funcționalități în funcție de **Impact** (Scăzut/Ridicat).
- Care dintre ele reprezintă un **Risc de Produs** mai mare dacă eșuează? De ce?
- Dacă serverul de testare nu funcționează timp de 3 zile, acesta este un **Risc de Produs** sau de **Proiect**? Justifică.

Partea C: Mindset-ul Critic (Creativitate)

Alege un obiect banal (ex: o telecomandă de televizor).

- Scrie 3 teste de tip **Happy Path** (utilizare normală).
- Scrie 3 teste de tip **Negative/Stress** (cum ai încerca să o „strici” sau să o „păcălești”).

Sfat de final de la echipa QualiAdept

Nu te teme să pui întrebări „prostești”. În testare, întrebarea „Dar dacă facem așa...?” este cea

care descoperă cele mai periculoase bug-uri. Ne vedem la Sesiunea 2, unde vom învăța cum să punem toate aceste idei într-un **Plan de Testare** profesionist!